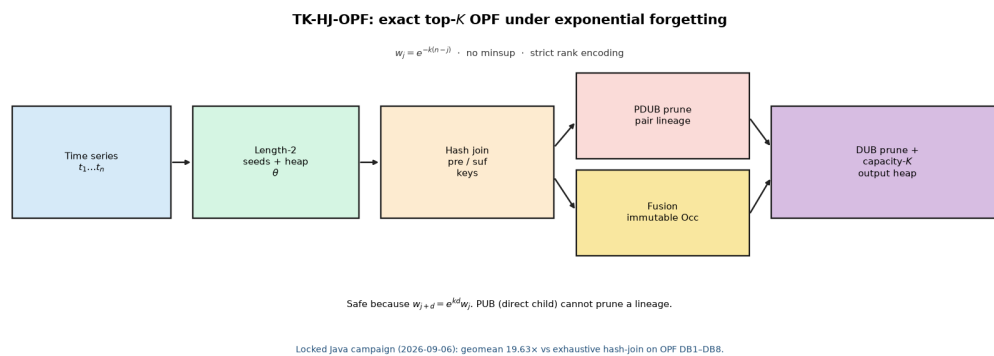


Graphical Abstract

TK-HJ-OPF: Exact Top- K Order-Preserving Pattern Mining under Exponential Forgetting

Khac-Chien Nguyen, Nhut-Minh Bui, Cong-Phat Tran



Highlights

TK-HJ-OPF: Exact Top- K Order-Preserving Pattern Mining under Exponential Forgetting

Khac-Chien Nguyen, Nhut-Minh Bui, Cong-Phat Tran

- Exact global top- K OPF mining without a minimum-support threshold.
- Endpoint-shift $w_{j+d} = e^{kd}w_j$ yields DUB and PDUB.
- An immediate-child PUB cannot prune a whole pair lineage.
- Hash-indexed fusion with immutable occurrence lists is exact.
- On OPF DB1-DB8, TK-HJ is 19.63x faster than exhaustive hash-join (geomean).

TK-HJ-OPF: Exact Top- K Order-Preserving Pattern Mining under Exponential Forgetting

Khac-Chien Nguyen^{a,*}, Nhut-Minh Bui^a, Cong-Phat Tran^a

^a*Faculty of Information Technology, People’s Police University, Ho Chi Minh City, Vietnam*

Abstract

Order-preserving (OP) mining encodes a time-series window by relative ranks, so trend shape is independent of amplitude. OPF-Miner weights each occurrence by exponential forgetting, but it still requires a user-specified minimum support. This paper formulates the exact, threshold-free top- K problem under that score (TK-OPF) and proposes TK-HJ-OPF. The method combines a capacity- K heap, an output-sensitive hash-indexed prefix-suffix join, immutable occurrence-list fusion, a descendant upper bound (DUB), and a pair-descendant upper bound (PDUB). Forgetting support is not anti-monotone under right extension: a surviving occurrence moves toward the present and is multiplied by e^{kd} . We prove the endpoint-shift identity $w_{j+d} = e^{kd}w_j$, derive DUB and PDUB from it, show by counterexample that an immediate-child pair bound (PUB) cannot prune a lineage, and establish join soundness, safe threshold raising, and end-to-end exactness under strict (tie-free) OP semantics. A Java 21 implementation is checked against exhaustive window enumeration (2400 randomized comparisons). On 6 September 2026 we locked a campaign on the official OPF DB1–DB8 files (SHA-256 matched). At $K = 50$, $\ell_{\max} = 12$, $k = 1/n$, five exact variants produced identical canonical top-50 sets, and full TK-HJ-OPF was 6.88–35.66 times faster than exhaustive hash-join (geometric mean 19.63). The same host and protocol on WDC-SILSO daily sunspots ($n = 72967$, tie-rich integers) gave a $15.02\times$ median speedup. Ablation shows that DUB supplies most of the consistent gain; PDUB is safe but not uniformly cheaper than DUB-only.

*Corresponding author.

Email address: `nkchienster@gmail.com` (Khac-Chien Nguyen)

Keywords: order-preserving pattern mining, exponential forgetting, top- K mining, hash join, branch-and-bound, time series

1. Introduction

Time-series analysis frequently depends on relative shape rather than absolute amplitude. Order-preserving (OP) matching encodes a window by the ranks of its values [1, 2]. The idea has grown into a mining family: frequent OP patterns [3], rules [4], approximation [5], contrast top- K [6], co-occurrence [7], outlying occurrences [8], and stability [9]. Order-preserving suffix trees (OPST) scale unweighted frequent, maximal, and closed mining [10, 11]. Zhou et al. review matching and indexing [12].

OPF-Miner addresses temporal relevance [13]: an occurrence ending at position j in a length- n series receives weight $w_j = e^{-k(n-j)}$. The method remains thresholded; its authors list automatic parameter setting and top- K OPF mining as future work. A naive conversion from minsup search to top- K is unsafe, because forgetting support is not anti-monotone under right extension.

This paper answers that gap *inside the classical fusion architecture*. TK-HJ-OPF (Top- K Hash-Join OPF) uses a hash-indexed prefix-suffix join, immutable occurrence lists, a capacity- K heap, and two forgetting-specific bounds (DUB, PDUB). The contributions are:

1. The TK-OPF problem: exact global top- K under OPF exponential forgetting, with a deterministic total order and no minsup (Definition 6).
2. The endpoint-shift identity (Lemma 1) and the bounds UB_d , DUB, PUB, PDUB, including a counterexample that PUB cannot prune a lineage (Example 1).
3. An output-sensitive hash-indexed fusion algorithm with immutable alignment, together with join soundness, safe pruning under threshold raising, and end-to-end exactness (Theorems 1–6).
4. A locked Java campaign on official OPF DB1–DB8: identical canonical top-50 sets across five exact modes, geometric-mean $19.63\times$ over exhaustive hash-join, and a negative ablation (PDUB safe but not uniformly profitable).

We do not claim the first top- K OP method (COPP-Miner optimizes contrast [6]), nor a replacement for unweighted OPST mining [11]. TK-HJ-

Table 1: Closest OP mining lines versus TK-HJ-OPF.

Work	Forgetting	Native top- K	Distinction
OPP-Miner [3]	no	no	unweighted, minsup
OPF-Miner [13]	yes	no	minsup predecessor
COPP-Miner [6]	no	yes	contrast score
SOPP-Miner [9]	no	downstream	stability
OPST mining [11]	no	no	unweighted index
TK-HJ-OPF	yes	yes	strict fusion

OPF answers: which K OP trends carry the greatest exponentially decayed evidence in one series, under strict (tie-free) windows.

2. Related work

OPP-Miner mines frequent OP patterns under minsup [3]. Later miners add rules, approximation, co-occurrence, outliers, and stability [4, 5, 7, 8, 9]. COPP-Miner is native top- K but scores class contrast [6]. OPF-Miner is the semantic predecessor and remains thresholded [13]. After an OPST is built, unweighted frequent/maximal/closed OP patterns can be reported in linear time in n [10, 11]; a static leaf count is not $\sum_j w_j$. Recency weighting outside OP mining includes OIP-Miner [14]. Top- K itemset miners such as TFP [15] motivate threshold raising; their monotonicities do not transfer. Hash-indexed pair discovery is classical [16, 17].

3. Preliminaries and problem

Notation is 1-based, matching OPF-Miner [13]. Companion code uses 0-based arrays.

Definition 1 (Time series). A time series is $\mathbf{t} = (t_1, \dots, t_n)$ with $t_i \in \mathbb{R}$. Input NaN and infinities are rejected; $+0$ and -0 are identified.

Definition 2 (Strict rank encoding). For a tie-free vector $\mathbf{z} = (z_1, \dots, z_m)$,

$$\text{norm}(\mathbf{z})_i = 1 + |\{h : z_h < z_i\}|. \quad (1)$$

If $\mathbf{z}$ contains a tie, the window is rejected. A length- m strict OP pattern is a permutation of $\{1, \dots, m\}$.

Definition 3 (Occurrences). A window of length m ending at j occupies $t_{j-m+1:j}$. For a pattern $\mathbf{p}$,

$$\text{Occ}(\mathbf{p}) = \{j \in \{m, \dots, n\} : \text{norm}(t_{j-m+1:j}) = \mathbf{p}\}. \quad (2)$$

Forgetting weights depend on the endpoint j .

Definition 4 (Forgetting weight and support). Let $k \geq 0$. The weight of ending position j is

$$w_j = e^{-k(n-j)}, \quad (3)$$

so $w_n = 1$. The forgetting support is

$$\text{fsup}(\mathbf{p}) = \sum_{j \in \text{Occ}(\mathbf{p})} w_j. \quad (4)$$

If $k = 0$, then $\text{fsup}(\mathbf{p}) = |\text{Occ}(\mathbf{p})|$.

Lemma 1 (Endpoint-shift identity). *For any valid endpoint j and integer $d \geq 0$ with $j + d \leq n$,*

$$w_{j+d} = e^{kd} w_j. \quad (5)$$

Proof. $w_{j+d} = e^{-k(n-j-d)} = e^{kd} e^{-k(n-j)} = e^{kd} w_j$. $\square$

Remark 1 (Non-anti-monotonicity). Extending an occurrence of length m starting at s to length $m + 1$ multiplies that occurrence's weight by e^k . The number of occurrences can only drop, but surviving mass becomes heavier, so fsup need not decrease.

Definition 5 (Normalized prefix and suffix). For a strict pattern $\mathbf{p}$ of length m ,

$$\text{pre}(\mathbf{p}) = \text{norm}(p_1, \dots, p_{m-1}), \quad \text{suf}(\mathbf{p}) = \text{norm}(p_2, \dots, p_m). \quad (6)$$

Two length- m patterns are *compatible* when $\text{suf}(\mathbf{p}) = \text{pre}(\mathbf{q})$. For a family F_m ,

$$\text{Comp}(F_m) = \{(\mathbf{p}, \mathbf{q}) \in F_m^2 : \text{suf}(\mathbf{p}) = \text{pre}(\mathbf{q})\}, \quad (7)$$

with $L_m = |F_m|$ and $J_m = |\text{Comp}(F_m)|$.

Definition 6 (TK-OPF). Given $\mathbf{t}$, $k \geq 0$, integer $K \geq 1$, and $2 \leq \ell_{\min} \leq \ell_{\max} \leq n$, let $\mathcal{P}$ be the set of observed strict patterns with length in $[\ell_{\min}, \ell_{\max}]$. Return the first $\min(K, |\mathcal{P}|)$ patterns in the total order: (i) larger fsup ; (ii) shorter length; (iii) lexicographically smaller rank sequence. No min-sup is supplied.

Table 2: Exact top-5 on the unified running series (8).

Rank	Pattern	Occ(p)	fsup
1	(2, 1)	{3, 4, 6, 7, 8, 10}	4.275265960
2	(1, 3, 2)	{3, 6, 10}	2.166905350
3	(3, 2, 1)	{4, 7, 8}	2.108360610
4	(1, 2)	{2, 5, 9}	1.960697042
5	(2, 1, 3)	{5, 9}	1.511368078

A capacity- K min-heap induces a threshold θ : $\theta = 0$ while the heap is not full, otherwise θ is the support of the current worst element. Insertions never decrease θ .

Running example.. The OPF series

$$\mathbf{t} = (15, 32, 29, 27, 34, 33, 25, 20, 28, 23), \quad n = 10, \quad k = 0.1, \quad K = 5 \quad (8)$$

is used throughout. For $\mathbf{p} = (1, 3, 2)$, $\text{Occ}(\mathbf{p}) = \{3, 6, 10\}$ and $\text{fsup}(\mathbf{p}) = e^{-0.7} + e^{-0.4} + 1 = 2.166905350$. Table 2 is the locked self-test.

4. Output-sensitive indexed fusion

Lemma 2 (Direct rank projection). *Let $\mathbf{p}$ be a permutation of $\{1, \dots, m\}$. Removing rank x sends every remaining rank $v > x$ to $v - 1$ and leaves $v < x$ unchanged. Thus $\text{pre}(\mathbf{p})$ and $\text{suf}(\mathbf{p})$ each take $O(m)$ time.*

Proof. Removing x removes exactly one smaller predecessor of every $v > x$. Removing p_m (resp. p_1) yields the prefix (resp. suffix). $\square$

Lemma 3 (Faithful compatibility key). *Under Definition 2, $\text{suf}(\mathbf{p}) = \text{pre}(\mathbf{q})$ iff the $m - 1$ overlapping positions are order-isomorphic.*

Proof. A strict rank vector canonically encodes a strict total order. $\square$

Theorem 1 (Join soundness and completeness). *Inserting every $\mathbf{q} \in F_m$ into a multimap keyed by $\text{pre}(\mathbf{q})$ and probing with $\text{suf}(\mathbf{p})$ emits exactly $\text{Comp}(F_m)$.*

Proof. Emitted pairs have equal keys, hence are compatible. Conversely, a compatible $\mathbf{q}$ lies in exactly the bucket probed by $\mathbf{p}$. $\square$

Expected hash work is $O(L_m + J_m)$ after keys exist; Lemma 2 costs $O(mL_m)$ if keys are not cached.

Algorithm 1 HJ-Compatible: emit $\text{Comp}(F_m)$

Require: family F_m of length- m strict patterns with cached pre / suf keys

```
1:  $H \leftarrow$  empty multimap
2: for each  $\mathbf{q} \in F_m$  do
3:   append  $\mathbf{q}$  to  $H[\text{pre}(\mathbf{q})]$ 
4: end for
5: for each  $\mathbf{p} \in F_m$  do
6:   for each  $\mathbf{q}$  in  $H.\text{lookup}(\text{suf}(\mathbf{p}))$  do
7:     emit  $(\mathbf{p}, \mathbf{q})$ 
8:   end for
9: end for
```

5. Forgetting-specific upper bounds

A *right-descendant at depth d* of a length- m pattern $\mathbf{p}$ is a length- $(m+d)$ pattern whose first m positions are order-isomorphic to $\mathbf{p}$.

Theorem 2 (Depth- d descendant bound). *For any right-descendant $\mathbf{x}$ of $\mathbf{p}$ at depth d ,*

$$\text{fsup}(\mathbf{x}) \leq \text{UB}_d(\mathbf{p}) := e^{kd} \sum_{\substack{o \in \text{Occ}(\mathbf{p}) \\ o \leq n-d}} w_o. \quad (9)$$

Proof. Every occurrence of $\mathbf{x}$ ending at j induces a unique prefix occurrence of $\mathbf{p}$ ending at $o = j - d$ with $o \leq n - d$. If S is the subset realizing $\mathbf{x}$, Lemma 1 gives $\text{fsup}(\mathbf{x}) = e^{kd} \sum_{o \in S} w_o$, which is at most (9). $\square$

Definition 7 (Descendant Upper Bound). For $D = \ell_{\max} - |\mathbf{p}|$,

$$\text{DUB}(\mathbf{p}) = \max_{0 \leq d \leq D} \text{UB}_d(\mathbf{p}). \quad (10)$$

Corollary 1 (Safe subtree pruning). *If the heap is full and $\text{DUB}(\mathbf{p}) < \theta$, no allowed right-descendant of $\mathbf{p}$ can belong to the final top- K . The inequality is strict so that support ties remain available for the secondary order.*

Proposition 1 (Efficient DUB evaluation). *Let $o_1 < \dots < o_s$ be sorted endpoints and $W_i = \sum_{h \leq i} w_{o_h}$. The active set in UB_d changes only when d crosses $n - o_i$. Between changes the weighted sum is constant while e^{kd} increases, so maxima occur at interval right endpoints. It suffices to evaluate $d = 0$, $d = D$, and breakpoints $d = n - o_i$.*

Theorem 3 (Immediate-child PUB). *Let $\mathbf{p}, \mathbf{q}$ be compatible length- m patterns and $\mathbf{r}$ any direct length- $(m+1)$ child. Then*

$$\text{fsup}(\mathbf{r}) \leq \text{PUB}(\mathbf{p}, \mathbf{q}) := \min\{e^k \text{fsup}(\mathbf{p}), \text{fsup}(\mathbf{q})\}. \quad (11)$$

Proof. A child occurrence ending at j maps injectively to a prefix occurrence of $\mathbf{p}$ at $j-1$ and a suffix occurrence of $\mathbf{q}$ at j . Lemma 1 gives $\text{fsup}(\mathbf{r}) \leq e^k \text{fsup}(\mathbf{p})$ and $\text{fsup}(\mathbf{r}) \leq \text{fsup}(\mathbf{q})$. $\square$

PUB is safe for the direct child only.

Example 1 (Naive PUB lineage pruning is incorrect). On (8) let $\mathbf{p} = (1, 4, 3, 2)$, $\text{Occ}(\mathbf{p}) = \{4\}$, $\mathbf{q} = (3, 2, 1, 4)$, $\text{Occ}(\mathbf{q}) = \{5\}$. Then $\text{fsup}(\mathbf{p}) = e^{-0.6} = 0.548812$, $\text{fsup}(\mathbf{q}) = e^{-0.5} = 0.606531$, the direct child $\mathbf{r} = (1, 4, 3, 2, 5)$ has support 0.606531, and $\text{PUB} = 0.606531$. The later descendant $\mathbf{x} = (1, 8, 7, 5, 10, 9, 4, 2, 6, 3)$ occurs at endpoint 10 with support 1.0. A temporary threshold 0.8 would drop the pair under PUB and lose $\mathbf{x}$.

Theorem 4 (Depth- d pair-descendant bound). *Let $\mathbf{x}$ be obtained from a compatible pair $(\mathbf{p}, \mathbf{q})$ by one direct child and $d \geq 0$ right extensions. Then*

$$\text{fsup}(\mathbf{x}) \leq \min\{\text{UB}_{d+1}(\mathbf{p}), \text{UB}_d(\mathbf{q})\}. \quad (12)$$

Proof. An occurrence of $\mathbf{x}$ ending at j realizes $\mathbf{p}$ at $j-(d+1)$ and $\mathbf{q}$ at $j-d$. Both maps are injective; apply Theorem 2. $\square$

Definition 8 (Pair-Descendant Upper Bound). With $D = \ell_{\max} - (m+1)$,

$$\text{PDUB}_D(\mathbf{p}, \mathbf{q}) = \max_{0 \leq d \leq D} \min\{\text{UB}_{d+1}(\mathbf{p}), \text{UB}_d(\mathbf{q})\}. \quad (13)$$

Corollary 2 (Safe pre-fusion pruning). *If the heap is full and $\text{PDUB}_D(\mathbf{p}, \mathbf{q}) < \theta$, the pair can be discarded before occurrence alignment.*

On Example 1, $\text{PDUB}_5 = 1.0$, so the pair survives any threshold up to 1.0. Once $\mathbf{p} = (1, 4, 3, 2)$ is materialized, $\text{DUB}(\mathbf{p}) = 1.0$ is compared with the live heap threshold.

Algorithm 2 TK-HJ-OPF

Require: series $\mathbf{t}$, decimal $k \geq 0$, integer K , $[\ell_{\min}, \ell_{\max}]$

```
1:  $F \leftarrow$  all length- $\ell_{\min}$  strict patterns with exact fsup
2:  $H \leftarrow$  capacity- $K$  heap on  $F$ ;  $\theta \leftarrow$  heap threshold
3:  $Front \leftarrow \{p \in F : H \text{ not full or } \text{DUB}(p) \geq \theta\}$ 
4: for  $m = \ell_{\min}, \dots, \ell_{\max} - 1$  do
5:    $J \leftarrow$  hash-join Comp( $Front$ ) ▷ Theorem 1
6:    $Next \leftarrow \emptyset$ 
7:   for each pair  $(\mathbf{p}, \mathbf{q}) \in J$  do
8:     if  $H$  full and  $\text{PDUB}(\mathbf{p}, \mathbf{q}) < \theta$  then
9:       continue
10:    end if
11:    for each child  $\mathbf{r}$  of exact fusion of  $(\mathbf{p}, \mathbf{q})$  do
12:      score  $\mathbf{r}$ ; possibly insert  $H$  and raise  $\theta$ 
13:      if  $|\mathbf{r}| < \ell_{\max}$  and  $(H \text{ not full or } \text{DUB}(\mathbf{r}) \geq \theta)$  then
14:        add  $\mathbf{r}$  to  $Next$ 
15:      end if
16:    end for
17:  end for
18:   $Front \leftarrow Next$ 
19: end for
20: return  $H$ 
```

6. The TK-HJ-OPF algorithm

Lemma 4 (Exact immutable fusion). *For a compatible pair, immutable occurrence alignment enumerates exactly the strict OP occurrences of each generated child, and summing w_j equals Definition 4.*

Proof. A child occurrence ending at j has a unique prefix endpoint $j - 1$ and suffix endpoint j . Conversely, an aligned compatible pair fixes the overlap order; comparing the first and last values completes the strict total order, or the window is rejected on a tie. $\square$

Worked trace.. On (8), length 2 inserts (2, 1) (support 4.275266) and (1, 2) (1.960697). For $\mathbf{p} = (1, 3, 2)$, $\text{suf}(\mathbf{p}) = (2, 1) = \text{pre}(3, 2, 1)$, so the join emits that pair. Fusion produces (1, 4, 3, 2) at endpoint 4 (0.548812) and (2, 4, 3, 1)

at 7 (0.740818). Once five patterns exist, $\theta = 1.511368$. The heap returns Table 2.

7. Correctness and complexity

Lemma 5 (Threshold monotonicity). *If θ_t is the heap threshold at a prune time and θ_f the threshold at termination, then $\theta_f \geq \theta_t$.*

Proof. The heap replaces its worst element only with a strictly better pattern under Definition 6, so the worst-in-heap support never decreases after the heap is full. $\square$

Theorem 5 (Safe pruning). *Any pair lineage pruned by PDUB and any subtree pruned by DUB contains no final top- K pattern.*

Proof. Corollaries 2 and 1 bound discarded patterns strictly below θ_t . Lemma 5 gives $\theta_f \geq \theta_t$. $\square$

Theorem 6 (Soundness and completeness). *Under strict OP semantics and exact real-valued support, Algorithm 2 returns exactly the deterministic top- K patterns in $[\ell_{\min}, \ell_{\max}]$.*

Proof. Soundness. Lemma 4: every heap insertion uses an exact occurrence list.

Completeness. Let $\mathbf{x}$ be a final top- K pattern of length $m + 1 > 2$, $\mathbf{p} = \text{norm}(x_1, \dots, x_m)$, $\mathbf{q} = \text{norm}(x_2, \dots, x_{m+1})$. Theorem 2 gives $\text{fsup}(\mathbf{x}) \leq \text{DUB}(\mathbf{p})$ and $\text{fsup}(\mathbf{x}) \leq \text{DUB}(\mathbf{q})$. Then $\theta_t \leq \theta_f \leq \text{fsup}(\mathbf{x})$, so neither parent is DUB-pruned. Theorem 1 emits the pair; $\text{PDUB} \geq \text{fsup}(\mathbf{x}) \geq \theta_t$, so the pair is not pruned. Lemma 4 generates $\mathbf{x}$. Induction from length- $\ell_{\min}$ seeds, plus a capacity- K heap, yields the global top K . $\square$

Let P_m be patterns surviving to generation m , $B_m \subseteq J_m$ pairs rejected by PDUB, $A(\mathbf{p}, \mathbf{q})$ alignment comparisons, S scored patterns, and C_{PD} , C_{DUB} bound-evaluation costs.

Theorem 7 (Expected work). *Ignoring JVM constants, expected work decomposes as*

$$\begin{aligned} &O\left(\sum_m (mL_m + L_m + J_m) + C_{\text{PD}} \right. \\ &\quad \left. + \sum_m \sum_{(\mathbf{p}, \mathbf{q}) \in J_m \setminus B_m} A(\mathbf{p}, \mathbf{q}) + C_{\text{DUB}} + S \log K\right). \end{aligned} \tag{14}$$

Equation (14) is search-space sensitive, not a polynomial in n alone. No claim of asymptotic global optimality is made.

Table 3: Locked environment.

Item	Value
CPU	Intel Core i5-8250U, 4c/8t, 1.60 GHz
Memory	32 GB
OS	Windows 11, build 26200, amd64
Java	Temurin OpenJDK 25.0.3; <code>javac -release 21</code>
JVM	<code>-Xms2g -Xmx8g</code> ; one JVM
Timing	<code>System.nanoTime()</code> around <code>mine()</code>
Central grid	5 warmups + 10 measured runs

Table 4: Official OPF-Miner series [18].

ID	Dataset	n
DB1	Amazon	5842
DB2	Russell 2000	8141
DB3	Nasdaq	12279
DB4	S&P 500	23046
DB5	NYSE	60000
DB6	CL.US	10305
DB7	HPQ.US	12075
DB8	GE.US	14058

8. Experimental verification

All numbers were measured on 6 September 2026 on one host. We do not reuse ARM64 wall-clock tables from an earlier laptop campaign.

Five exact modes: `hjtopk` (exhaustive join, no search pruning), `tk-no-bounds`, `tk-no-dub` (PDUB only), `tk-no-pdub` (DUB only), `tk` (both). Official series SHA-256 match the OPF-Miner suite [18] (Table 4). Geometric means follow Fleming and Wallace [19].

8.1. Exactness

Java `tk/hjtopk` and the single-file companion return Table 2 (tolerance 10^{-10}). `RandomizedSelfTest` (seed 20260818) completed 2400 exact comparisons against window enumeration. Canonical SHA-256 of the top-50 set is identical across all five modes on all eight official files.

Table 5: Central campaign, $K = 50$, $\ell_{\max} = 12$, $k = 1/n$. Median ms, ten measured runs after five warmups.

Dataset	n	HJ	DUB-only	Full TK	HJ/TK	HJ/DUB
Amazon	5842	69.941	1.859	1.961	35.66×	37.62×
Russell 2000	8141	64.229	2.542	2.669	24.07×	25.27×
Nasdaq	12279	89.371	3.349	3.508	25.48×	26.68×
S&P 500	23046	146.488	5.622	6.135	23.88×	26.05×
NYSE	60000	96.432	14.676	14.026	6.88×	6.57×
CL.US	10305	49.993	2.318	2.399	20.84×	21.56×
HPQ.US	12075	53.981	2.801	2.895	18.64×	19.27×
GE.US	14058	50.950	3.199	3.219	15.83×	15.93×
Geomean					19.63×	20.29×

8.2. Central DB1-DB8

Table 5 reports $K = 50$, $\ell_{\max} = 12$, $k = 1/n$. Full TK-HJ-OPF is 6.88–35.66× faster than exhaustive HJ (geomean 19.63). DUB-only is fastest on seven of eight datasets; full TK-HJ is slightly faster on NYSE. PDUB is safe but not uniformly cheaper than DUB-only.

On Amazon the IQR of full TK is [1.878, 2.521] ms versus [63.541, 83.517] ms for HJ. Figures 1 and 2 show the five-mode runtime (log scale) and per-dataset HJ/TK speedup.

The principal gain is pruning, not hashing alone. Table 6 records median pair-discovery and alignment counters at the same central cell. Exhaustive HJ on Amazon emits 20878 pair attempts and 39128 compatible pairs, versus 84 and 232 under full TK-HJ-OPF; aligned occurrence comparisons fall from 261668 to 95480. PDUB prunes 79 pairs and DUB prunes 105 child branches on that series. No-bound variants remain close to exhaustive HJ in wall-clock (Table 5 and `timing.csv`), so the speedup is explained by raising θ and discarding hopeless regions.

8.3. Sensitivity

Sweeps ran in the same JVM after the central grid; they support trends, not a second locked table. On Amazon at $\ell_{\max} = 12$, HJ/TK falls from 61.3× at $K = 10$ to 8.0× at $K = 500$ (Fig. 3). Exhaustive HJ on Amazon

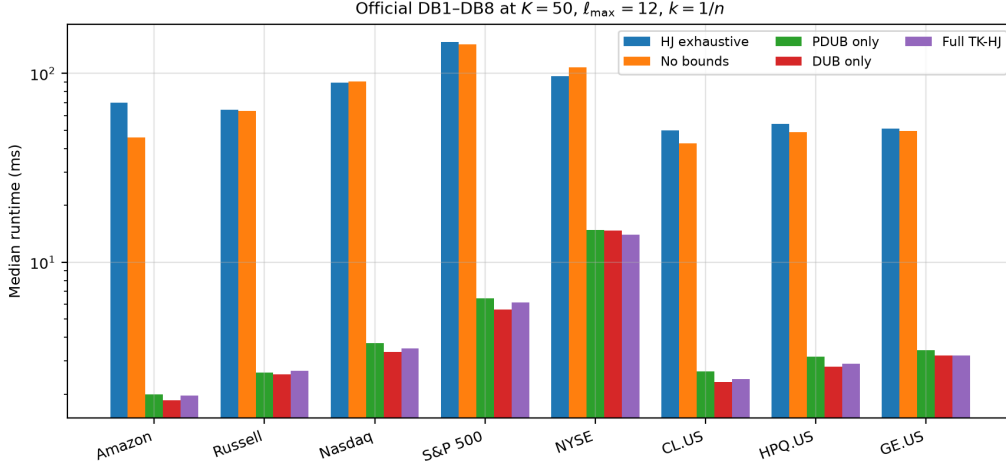


Figure 1: Median runtime of five exact variants on official DB1-DB8 at $K = 50$, $\ell_{\max} = 12$, $k = 1/n$. Logarithmic y -axis.

grows with $\ell_{\max}$ (10.7 ms at 8, 37.7 at 12, 92.0 at 16) while full TK stays near 2 ms. Varying $k \in \{0.25, 0.5, 1, 2, 4\}/n$ at $K = 50$ did not change the runtime regime on Amazon or GE.US.

8.4. Public tie-rich series (SILSO)

To step outside the financial OPF suite we ran the same host, JVM flags, and protocol (5 warmups + 10 measured, $K = 50$, $\ell_{\max} = 12$, $k = 1/n$) on WDC-SILSO daily total sunspot number v2.0 [20]. Missing markers (SSN < 0) were dropped, leaving $n = 72967$ integer counts. Strict encoding rejects tied windows, so this series stresses the tie-rejection path.

Median milliseconds: exhaustive HJ 191.797, DUB-only 12.325, full TK-HJ 12.768 (HJ/TK = 15.02 $\times$). DUB-only is again slightly faster than full TK-HJ. A Windows Working-Set sample of the *whole* public-campaign JVM (all three modes in one process, `-Xms2g -Xmx8g`) peaked at 1303.5 MB. That number is process-level, not a per-algorithm OS Peak RSS, and is not compared with Table 5.

8.5. Memory reporting

Campaign CSVs record diagnostic in-process JVM heap (`totalMemory - freeMemory`) in column `peak_heap_mb`. Those values are *not* OS Peak RSS

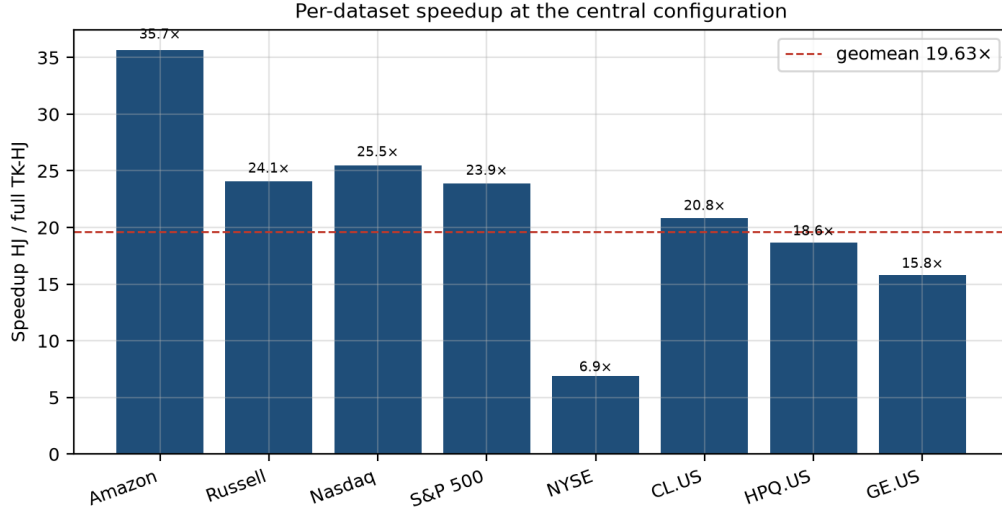


Figure 2: Per-dataset speedup of full TK-HJ-OPF over exhaustive HJ at the central configuration. Dashed line: geometric mean $19.63\times$.

and were collected in one long-lived JVM, so they must not be read as a cross-algorithm memory ranking. A per-mode RSS campaign in a fresh process remains future work.

8.6. What is not claimed

No per-algorithm OS Peak RSS. No FRED timings on this host. No OPST runtime. No architecture-independent speedup.

9. Discussion

TK-HJ-OPF is a weighted ranking method, not a general replacement for OP mining. Unweighted maximal or closed discovery at very large scale is the natural home of OPST indexing [11]. Class discrimination is the home of COPP-Miner [6]. Stability of occurrence distributions is the home of SOPP-Miner [9]. TK-HJ-OPF answers: which K OP trends carry the greatest exponentially decayed evidence in one series, under strict windows.

The theoretical core is that a surviving occurrence becomes heavier under right extension (Lemma 1). DUB repairs subtree pruning; PUB is valid only for a direct child (Example 1); PDUB repairs pair-lineage pruning. The experiments add a second lesson: a safe earlier bound is not necessarily

Table 6: Median search-space counters at $K = 50$, $\ell_{\max} = 12$, $k = 1/n$. Pair attempts and compatible pairs are hash-join events; aligned checks are occurrence-list comparisons.

Dataset	HJ pairs	HJ aligned	TK pairs	TK aligned	PDUB prunes	DUB prunes
Amazon	20878	261668	84	95480	79	105
Russell 2000	27203	415273	84	138862	70	106
Nasdaq	35427	663551	82	206420	74	99
S&P 500	54459	1181569	81	374157	71	108
NYSE	27873	2778106	81	934774	95	104
CL.US	19036	316928	78	135185	68	105
HPQ.US	20077	377322	77	162972	61	112
GE.US	16697	330778	83	156966	77	107

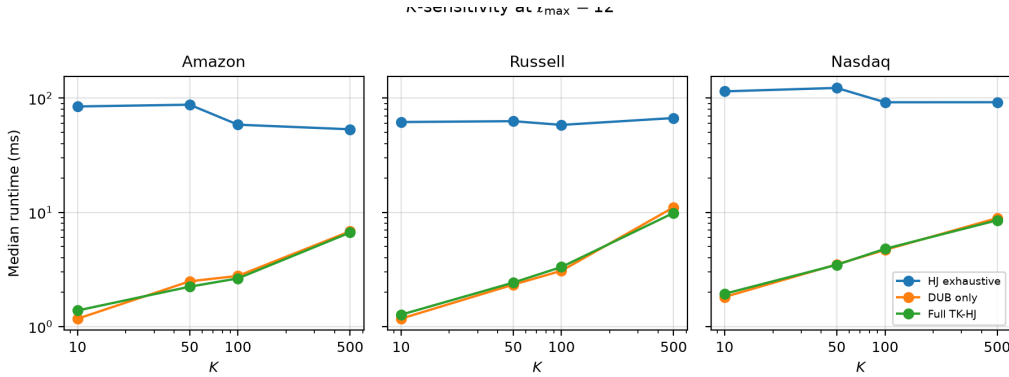


Figure 3: K -sensitivity on Amazon, Russell 2000, and Nasdaq at $\ell_{\max} = 12$ (log-log). Exhaustive HJ is comparatively insensitive to K ; full TK-HJ-OPF and DUB-only become slower as the heap threshold rises more slowly.

faster. Unconditional PDUB reduces alignment counts (Table 6) and can still lose to DUB-only in wall-clock (Table 5). A practical next step is an adaptive rule that compares expected alignment cost with expected PDUB cost from list sizes, depth, and current θ .

10. Threats to validity

Novelty is limited to exact global top- K under OPF forgetting with the proposed fusion bounds; we do not claim the first scalable OP method. A keyword search (IEEE Xplore / arXiv / TKDE, 6–7 September 2026; queries in NOVELTY_SEARCH.md) found OPF-Miner (minsup), COPP-Miner (contrast top- K), OPST unweighted mining, and OIP-Miner (non-OP forgetting), but

no published exact global top- K OPF fusion miner. That log is not a non-existence proof.

Mathematics assumes exact reals; Java uses IEEE-754 `Math.exp` with a conservative ULP prune guard. One x86 laptop and $\ell_{\max} \leq 16$ do not establish architecture-independent or unrestricted-length scalability. Official files are financial; SILSO is tie-rich integers under strict rejection.

11. Conclusion

TK-HJ-OPF solves exact threshold-free top- K OPF inside fusion: hash-indexed compatible pairs, immutable alignment, and DUB/PDUB derived from $w_{j+d} = e^{kd}w_j$. Example 1 shows that PUB cannot prune a lineage. On this host, at $K = 50$ and $\ell_{\max} = 12$, full TK-HJ-OPF is a $19.63\times$ geometric-mean improvement over exhaustive hash-join on DB1–DB8, with identical canonical top-50 sets, and $15.02\times$ on SILSO ($n = 72967$). DUB is the consistent accelerator and unconditional PDUB is not uniformly profitable. Future work includes adaptive PDUB activation, streaming TK-OPF, weighted OPST indexing, and a dedicated per-mode OS RSS campaign.

Data and code

Java 21 sources, the campaign runner, `timing.csv`, and `canonical.tsv` are in the companion repository <https://github.com/Porcupine1805/TK-HJ-OPF>. Official OPF-Miner series are not redistributed; SHA-256 checksums are in `DATASETS.md`.

CRedit authorship contribution statement

Khac-Chien Nguyen: conceptualization, methodology, software, validation, writing — original draft, writing — review and editing (corresponding author). Nhut-Minh Bui: methodology, software, validation, writing — review and editing. Cong-Phat Tran: methodology, software, validation, data curation, writing — review and editing. Roles can be revised by the authors before journal submission.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] J. Kim, P. Eades, R. Fleischer, S.-H. Hong, C. S. Iliopoulos, K. Park, S. J. Puglisi, T. Tokuyama, Order-preserving matching, *Theoretical Computer Science* 525 (2014) 68–79. doi:[10.1016/j.tcs.2013.10.006](https://doi.org/10.1016/j.tcs.2013.10.006).
- [2] M. Crochemore, C. S. Iliopoulos, T. Kociumaka, M. Kubica, A. Langiu, S. P. Pissis, J. Radoszewski, W. Rytter, T. Walen, Order-preserving incomplete suffix trees and order-preserving indexes, in: *SPIRE*, Vol. 8214 of *LNCS*, 2013, pp. 84–95. doi:[10.1007/978-3-319-02432-5_13](https://doi.org/10.1007/978-3-319-02432-5_13).
- [3] Y. Wu, Q. Hu, Y. Li, L. Guo, X. Zhu, X. Wu, Opp-miner: Order-preserving sequential pattern mining for time series, *IEEE Transactions on Cybernetics* 53 (5) (2023) 3288–3300. doi:[10.1109/TCYB.2022.3169327](https://doi.org/10.1109/TCYB.2022.3169327).
- [4] Y. Wu, X. Zhao, Y. Li, L. Guo, X. Zhu, P. Fournier-Viger, X. Wu, Opr-miner: Order-preserving rule mining for time series, *IEEE Transactions on Knowledge and Data Engineering* 35 (11) (2023) 11722–11735. doi:[10.1109/TKDE.2022.3224963](https://doi.org/10.1109/TKDE.2022.3224963).
- [5] Y. Li, J. Liu, Y. Guo, J. Liu, Y. Wu, Aop-miner: Approximate order-preserving pattern mining for time series, in: *2022 IEEE International Conference on Knowledge Graph (ICKG)*, 2022. doi:[10.1109/ICKG55886.2022.00026](https://doi.org/10.1109/ICKG55886.2022.00026).
- [6] Y. Wu, Y. Meng, Y. Li, L. Guo, X. Zhu, P. Fournier-Viger, X. Wu, Copp-miner: Top-k contrast order-preserving pattern mining for time series classification, *IEEE Transactions on Knowledge and Data Engineering* 36 (6) (2024) 2372–2387. doi:[10.1109/TKDE.2023.3321749](https://doi.org/10.1109/TKDE.2023.3321749).
- [7] Y. Wu, Z. Wang, Y. Li, Y. Guo, H. Jiang, X. Zhu, X. Wu, Co-occurrence order-preserving pattern mining with keypoint alignment for time series, *ACM Transactions on Management Information Systems* 15 (2) (2024) 1–27. doi:[10.1145/3658450](https://doi.org/10.1145/3658450).
- [8] Y. Wu, S. Lou, Y. Li, L. Guo, P. Fournier-Viger, X. Wu, Outo-miner: Detecting outlying occurrences in maximal frequent order-preserving patterns in time series, *Information Sciences* 720 (2025) 122497. doi:[10.1016/j.ins.2025.122497](https://doi.org/10.1016/j.ins.2025.122497).

- [9] Y. Zhang, Y. Wu, Y. Li, Sopp-miner: Stable distribution order-preserving pattern mining for time series, *IEEE Transactions on Knowledge and Data Engineering* 38 (8) (2026) 5406–5419. [doi:10.1109/TKDE.2026.3692778](https://doi.org/10.1109/TKDE.2026.3692778).
- [10] L. Li, W. Zuba, G. Loukides, S. P. Pissis, M. Matsangidou, Scalable order-preserving pattern mining, in: *2024 IEEE International Conference on Data Mining (ICDM)*, 2024, pp. 211–220. [doi:10.1109/ICDM59182.2024.00028](https://doi.org/10.1109/ICDM59182.2024.00028).
- [11] L. Li, W. Zuba, G. Loukides, S. P. Pissis, M. Matsangidou, Order-preserving pattern mining and comparison via indexing, *IEEE Transactions on Knowledge and Data Engineering* (2026) 1–18 [doi:10.1109/TKDE.2026.3710634](https://doi.org/10.1109/TKDE.2026.3710634).
- [12] Y. Zhou, F. Wang, Y. Tang, Order-preserving pattern matching: Review, *IEEE Transactions on Knowledge and Data Engineering* 38 (3) (2026) 1885–1904. [doi:10.1109/TKDE.2025.3649808](https://doi.org/10.1109/TKDE.2025.3649808).
- [13] Y. Li, C. Ma, R. Gao, Y. Wu, J. Li, W. Wang, X. Wu, Opf-miner: Order-preserving pattern mining with forgetting mechanism for time series, *IEEE Transactions on Knowledge and Data Engineering* 36 (12) (2024) 8981–8995. [doi:10.1109/TKDE.2024.3438274](https://doi.org/10.1109/TKDE.2024.3438274).
- [14] Y. Wu, S. Tian, Y. Li, J. Liu, J. Li, W. Wang, X. Wu, Oip-miner: One-off incremental sequential pattern mining with forgetting factor, *Science China Information Sciences* 68 (2025) 209101. [doi:10.1007/s11432-024-4496-9](https://doi.org/10.1007/s11432-024-4496-9).
- [15] J. Wang, J. Han, Y. Lu, P. Tzvetkov, Tfp: An efficient algorithm for mining top-k frequent closed itemsets, *IEEE Transactions on Knowledge and Data Engineering* 17 (5) (2005) 652–664. [doi:10.1109/TKDE.2005.81](https://doi.org/10.1109/TKDE.2005.81).
- [16] J. S. Park, M.-S. Chen, P. S. Yu, An effective hash-based algorithm for mining association rules, in: *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, 1995, pp. 175–186. [doi:10.1145/223784.223813](https://doi.org/10.1145/223784.223813).
- [17] C. Balkesen, J. Teubner, G. Alonso, M. T. Özsu, Main-memory hash joins on multi-core cpus: Tuning to the underlying hardware, in: *2013*

- IEEE 29th International Conference on Data Engineering (ICDE), 2013, pp. 362–373. doi:10.1109/ICDE.2013.6544839.
- [18] Y. Wu, Y. Li, C. Ma, R. Gao, [OPF-Miner Algorithms and DataSets](#), GitHub repository, accessed 18 August 2026 (2024).
URL <https://github.com/wuc567/Pattern-Mining/tree/master/OPF-Miner>
- [19] P. J. Fleming, J. J. Wallace, How not to lie with statistics: The correct way to summarize benchmark results, *Communications of the ACM* 29 (3) (1986) 218–221. doi:10.1145/5666.5673.
- [20] WDC-SILSO, Royal Observatory of Belgium, [Daily total sunspot number, version 2.0](#), accessed 18 August 2026; CC BY-NC 4.0 (2026).
doi:10.24414/qnza-ac80.
URL https://www.sidc.be/SILSO/DATA/SN_d_tot_V2.0.csv